

Healthcare Insurance Assistant — Architecture & Design

What This Chatbot Does

This is a chat assistant that answers questions about a health insurance plan by reading the plan's own PDF documents (Evidence of Coverage, Summary of Benefits, Policy Manual) and quoting from them, rather than making up answers.

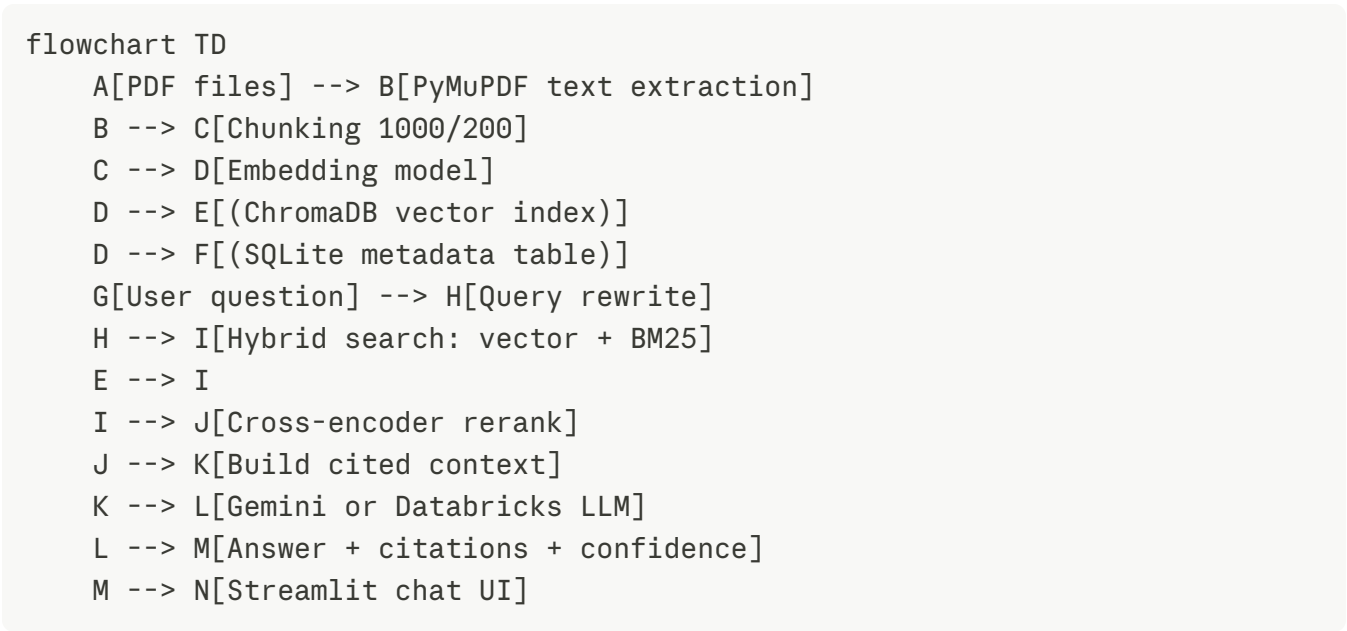
The core idea is called **Retrieval-Augmented Generation (RAG)**: instead of asking an AI model to answer purely from what it memorized during training, the app first *retrieves* the most relevant paragraphs from the real documents, then asks the AI to *generate* an answer using only those paragraphs. This is why the chatbot can say "the deductible is \$500" and point to the exact page it came from, and why it says "I could not find this information" instead of guessing when the documents don't cover a topic.

Three things make this trustworthy for an insurance use case:

- **Every answer is grounded** in the actual document text, never the model's general knowledge.
- **Every answer is cited** with a document name and page number.
- **Every retrieved chunk has a relevance score**, so low-confidence answers are visible, not hidden.

High-Level Architecture

The app runs two separate flows: a one-time (or occasional) **ingestion** flow that reads PDFs into a searchable index, and a **chat** flow that runs on every question.



Both flows share the same embedding model, which matters: a question is only comparable to a stored chunk if both were turned into vectors by the *same* model. Switching embedding providers requires re-running ingestion so both sides match again.

The app is intentionally split into small, single-purpose modules (one file per concern) rather than one large script, so each piece can be tested, replaced, or reused independently. The file-by-file sections below walk through every module in that pipeline.

Files: Reading and Splitting Documents

File	What it does
<code>ingestion/pdf_loader.py</code>	Opens each PDF with PyMuPDF and pulls out the plain text of every page, one page at a time. Skips blank or unreadable pages instead of crashing. Guesses a document type ("Evidence of Coverage", "Summary of Benefits", "Policy Manual") from the filename, used later for citations.

File	What it does
<code>chunking/chunker.py</code>	Splits each page's text into ~1000-character pieces with 200 characters of overlap between pieces, using LangChain's <code>RecursiveCharacterTextSplitter</code> . Each piece keeps a unique id, its source file name, page number, and document type, so it can be cited later.

Why split by page first, then by 1000 characters? Page numbers only make sense as citations if the text is never mixed across pages, so chunking always runs *within* one page's text, never across a page boundary.

Files: Turning Text Into Vectors (Embeddings)

An embedding is a list of numbers that represents the *meaning* of a piece of text, so two sentences worded differently but meaning the same thing end up numerically close together. This is what lets the chatbot find the right chunk even when the question's wording differs from the document's wording.

File	What it does
<code>embeddings/base.py</code>	Defines the shape every embedding provider must follow (<code>embed_documents</code> , <code>embed_query</code>), so the rest of the app never needs to know which one is active.
<code>embeddings/local_embeddings.py</code>	Free, offline embeddings using <code>sentence-transformers</code> (model: <code>all-MiniLM-L6-v2</code>), downloaded once from Hugging Face. No API key, no cost, but needs internet on first run to fetch the model.
<code>embeddings/gemini_embeddings.py</code>	Hosted embeddings via the Gemini API (<code>gemini-embedding-001</code>). Includes batching, retry with backoff, and a rate limiter that paces requests to stay under Gemini's free-tier quota.

File	What it does
<code>embeddings/databricks_embeddings.py</code>	Embeddings via a Databricks Model Serving endpoint. Authenticates through the Databricks SDK, which needs no API key at all when this app runs inside a Databricks workspace.
<code>embeddings/embedding_service.py</code>	The single door the rest of the app calls through. Reads one setting (<code>EMBEDDING_PROVIDER</code>) and hands back whichever provider above is configured, built once and reused.

Because every provider implements the same shape, switching providers is a one-line configuration change, not a code change.

Files: Storing and Searching Chunks

File	What it does
<code>vector_store/chroma_manager.py</code>	Wraps ChromaDB, a lightweight on-disk database built to answer "which stored vectors are closest to this new vector" in milliseconds. Handles inserting chunks, deleting a document's old chunks before re-adding a changed version, and running similarity search.
<code>vector_store/metadata_table.py</code>	A SQLite database that mirrors every chunk (id, document name, page, text, timestamp) in a plain queryable table, plus a second table tracking a content hash per source file so re-ingestion can skip files that haven't changed.

Why two stores instead of one? ChromaDB is built for fast similarity search, not for questions like "show me every chunk from page 3" or "has this file already been processed." SQLite handles that structured, auditable side cheaply, the way a real deployment would split a vector index from a system-of-record table.

Files: Answering the Question

File	What it does
<code>rag_pipeline/retrieval_service.py</code>	Embeds the question, asks ChromaDB for the top matches, converts ChromaDB's distance into a 0-1 relevance score, and optionally blends in a keyword score (BM25) so exact terms like plan codes aren't missed by pure meaning-matching.
<code>rag_pipeline/reranker.py</code>	Optionally re-scores the top candidates with a small cross-encoder model, which reads the question and one chunk together for a more accurate relevance judgment than similarity search alone. Falls back cleanly if the model can't be loaded.
<code>rag_pipeline/query_rewriter.py</code>	Rewrites a follow-up question ("what about for my spouse?") into a standalone one before searching, using the conversation history. Skipped on the first question, where there's nothing to rewrite against.
<code>rag_pipeline/memory.py</code>	Keeps the last few question/answer turns in memory, so follow-ups have context and the app doesn't resend the whole conversation forever.
<code>rag_pipeline/guardrails.py</code>	Blocks obvious prompt-injection attempts ("ignore your instructions") before they reach the model, and checks after the fact whether an answer's wording actually overlaps with the retrieved text.
<code>rag_pipeline/prompt_templates.py</code>	The instructions given to the AI model: answer only from the provided context, cite every claim, and say a fixed sentence when the answer isn't in the documents.
<code>rag_pipeline/llm_service.py</code>	Reads one setting (<code>LLM_PROVIDER</code>) and builds either a Gemini or a Databricks chat model, the same one-door pattern used for embeddings.

File	What it does
<code>rag_pipeline/rag_pipeline.py</code>	The conductor. Runs retrieve -> rerank -> build context -> generate -> check grounding for every question, and retries the model call before falling back to a plain "service unavailable" message instead of crashing.

Files: The Chat Screen and Supporting Pieces

File	What it does
<code>frontend/app.py</code>	The Streamlit chat screen. Only a chat box, no upload button by design; auto-ingests documents on first load if the index is empty; shows each answer with expandable sources, a confidence bar, and thumbs up/down feedback.
<code>config/settings.py</code>	Every tunable value in one place (chunk size, top-K, which provider, model names), read from a <code>.env</code> file. This is the only file that reads secrets, so swapping how secrets are stored later means changing one file.
<code>utils/logging_utils.py</code>	One consistently formatted logger the whole app shares, so errors show which module and what happened.
<code>utils/mlflow_tracking.py</code>	Logs every question as an MLflow run (model used, response time, number of chunks, retrieved sources) to a local folder, browsable with <code>mlflow ui</code> .
<code>utils/feedback_logger.py</code>	Appends each thumbs up/down click to a local file, for later review of which answers members found unhelpful.

File	What it does
<code>scripts/ingest.py</code>	Ties ingestion together: checks each PDF's content hash, skips unchanged files, and only re-processes what's new or edited. Runs both as a one-time command and automatically from the Streamlit app.

Files: Automated Tests

File	What it checks
<code>conftest.py</code>	Shared setup: a fake, hash-based embedding provider and a temporary folder for every test, so the test suite never touches real API keys, the internet, or the real project data.
<code>tests/test_ingestion.py</code>	PDF text extraction, document-type guessing, and that a corrupt file is skipped rather than crashing the whole batch.
<code>tests/test_chunking.py</code>	Chunk sizing, metadata (page, file, document type) staying attached to each piece.
<code>tests/test_retrieval.py</code>	Top-K limiting and score-threshold filtering behave correctly.
<code>tests/test_reranker.py</code>	Locks in the confidence-score bug fix: cross-encoder output must be normalized into 0-1, not left as a raw, unbounded number.
<code>tests/test_provider_switching.py</code>	Choosing Gemini vs. Databricks builds the right client, and that going fully keyless (both switches set to Databricks) never requires a Gemini key.
<code>tests/test_rag_pipeline.py</code>	Citation formatting, the exact fallback sentence when nothing relevant is found, prompt-injection blocking, and graceful degradation when the chat model keeps failing.

All 27 tests run in a few seconds with no network access, which is what makes them safe to run before every push.

Why These Models and Strategies

Chunk size 1000 / overlap 200. Small enough that 5 retrieved chunks still fit comfortably in a prompt; large enough to hold a complete thought ("the deductible is \$500 per member"). The 200-character overlap protects against a key sentence landing exactly on a chunk boundary and being split into two useless halves.

Embedding model choice, by provider:

Provider	Model	Why
Local	all-MiniLM-L6-v2	Free, offline, small enough to run on a laptop CPU; the standard default for prototyping without an API key.
Gemini	gemini-embedding-001	Hosted, needs only the same key already used for chat; the only option that works when a network blocks the local model's download.
Databricks	databricks-gte-large-en	Runs inside the same workspace as the data, no external network hop, can be fully keyless.

Hybrid search (vector + BM25). Vector search finds *meaning* matches ("how much before coverage starts" -> "deductible") but can miss exact rare terms like plan codes or dollar figures. BM25 is a classic keyword-frequency method that catches exactly those. Blending both (70% vector / 30% keyword) covers each method's blind spot.

Reranking with a cross-encoder. Vector search compares a question and a chunk that were each turned into a vector *separately*, which is fast but approximate. A cross-encoder reads the question and one candidate chunk *together* in a single pass, giving a more accurate relevance judgment. It's only run on the top ~15 candidates, not the whole index, because it's too slow to run on everything.

Why the LLM never answers from memory. The system prompt explicitly forbids using outside knowledge, and if retrieval finds nothing relevant, the app never even calls the model, returning the fixed "not found" sentence directly. This is the single biggest lever against hallucination in a domain where a wrong answer about coverage has real consequences.

How This Compares to Industry-Standard RAG Chatbots

Practice	This app	Industry-standard production system
Grounded, cited answers	Yes -- required by the system prompt	Yes, this is table stakes for any regulated-domain chatbot
Hybrid search + reranking	Yes	Yes, this exact pattern (vector + keyword, then cross-encoder) is the current standard
Pluggable model providers	Yes (local/Gemini/Databricks)	Yes, avoiding vendor lock-in is standard practice
Observability	Local MLflow logging only	A production system exports traces to a shared platform (Datadog, Grafana, or a hosted MLflow) with alerting
Vector store	ChromaDB, single file on disk	A managed, distributed vector database (Databricks Vector Search, Pinecone, pgvector) for scale and high availability
Evaluation	Manual, ad hoc testing during this session	A curated set of test questions with expected answers/citations, run automatically to catch regressions
Guardrails	Regex-based prompt-injection denylist	A dedicated content-safety model or service, layered with the same regex checks
Access control	None -- anyone who can reach the app can ask anything	Authentication (SSO/OAuth) and per-user audit logging, especially for health data
Deployment	Manual <code>streamlit run</code>	CI/CD pipeline, containerized, autoscaled, with a real uptime target

The retrieval and generation logic (the actual "RAG" core) already follows current best practice. What's missing is the operational layer around it -- the part that turns a working prototype into something a company can run unattended and be held accountable for.

Pain Points Found During Development and Testing

These are real issues hit while building and running this exact app, not a generic checklist.

1. **The reranker showed nonsensical confidence scores.** A live test returned "-480% confidence" because the cross-encoder's raw, unbounded score was used directly instead of being normalized. Found only by actually running the app, not by the test suite (fixed, and now covered by `test_reranker.py`).
2. **Free-tier rate limits hit twice, in two different places.** Gemini's embedding quota (~100/min) and its chat quota (5/min) are separate limits with separate failure modes; the chat quota crashed the app with a raw exception before a retry-and-fallback was added.
3. **Gemini model names go stale.** `gemini-2.0-flash` returned a "no longer available" error mid-session; the default model name is a config value that can silently break.
4. **A sandboxed network blocked the free local embedding model.** `huggingface.co` was unreachable in one test environment, forcing a fallback to hosted embeddings there -- a working local setup is not guaranteed to work in every environment.
5. **A transitive dependency upgrade broke a working setup.** Installing a newer Databricks integration package silently pulled in an incompatible `langchain-core` and a newer `mlflow` that refused the local file-store backend -- discovered only after tests started failing.
6. **The metadata SQLite file grew large relative to its content.** Storing every embedding vector twice (once in ChromaDB, once as JSON text in SQLite) added tens of megabytes for a few hundred chunks.
7. **The "confidence" score is a heuristic, not a calibrated probability.** Even after fixing the normalization bug, it reflects relative relevance, not "percent chance this answer is correct" -- a user could reasonably over-trust it.
8. **Local setup friction on a fresh machine.** Missing `python3-venv`, no `nano`, `python` vs `python3`, and `~/.local/bin` not on `PATH` each caused a stall during first-time setup on a real machine.
9. **A malicious PDF is not defended against.** The guardrails check the user's question for injection attempts, but nothing currently checks whether an *ingested document itself* contains text designed to manipulate the model.

Improvement Roadmap

Pain point	Improvement
Unnormalized reranker scores	Fixed already; add a small eval set that would have caught this before a user did
Rate limits crashed answers	Add the same rate-limit pacing already built for embeddings to the chat call, and show wait time in the UI instead of a generic error
Model names go stale	Query the provider's list-models endpoint at startup and warn (or auto-fallback) if the configured model is unavailable
Local embeddings blocked in some networks	Document it (done) and add a startup check that tests reachability and suggests the right provider automatically
Dependency upgrade broke the app	Add CI (GitHub Actions) that installs from a lockfile and runs the full test suite on every push, before anything reaches a real user
Oversized metadata store	Store a reference or hash in SQLite instead of the full duplicated vector; SQLite already isn't the search engine
Confidence isn't calibrated	Relabel it as "relevance" in the UI, and build a small labeled evaluation set to measure real answer accuracy over time
Local setup friction	Add a single setup script that checks for <code>venv</code> , <code>pip</code> , and <code>PATH</code> issues and fixes or explains each one
No defense against a poisoned document	Add a content-safety pass over newly ingested text, separate from the user-input guardrail, before it ever reaches the model
No production access control	Add authentication (SSO/OAuth) and per-question audit logging before handling real member data

Pain point	Improvement
No production-grade vector store	Swap ChromaDB for Databricks Vector Search or another managed store when moving beyond a single-machine deployment (already designed as a drop-in swap)

None of these require a rewrite. The codebase was built with each of these seams already isolated (one file per provider, one settings file, one retrieval service), which is exactly what makes each row in this table a contained change rather than a redesign.